Text classification
Sentiment analysis using spaCy and scikit learn

Subject: Reviews of television series.

In this practical session, we will explore machine learning models for NLP applications ; specifically, we will train a classifier for sentiment analysis on a French dataset of movie reviews. For these exercises, we will make use of Python (v3) and several modules for data processing and machine learning: numpy, scipy, scikit-learn, pandas *and* spacy.

If you want to use your own computer instead of colab., you will need to make sure the modules are installed (e.g. using the command *pip*).

Tasks to do:

We'll go through the following stages of an NLP machine learning pipeline, using sentiment classification as an application:

- data preprocessing (tokenization)
- feature extraction
- model training
- evaluation

The dataset we analyze:

As a dataset, we'll be using a set of reviews for television series in French, extracted from the website allocine.fr.

The dataset consists of the text of the review, as well as a sentiment label (positive or negative).

The training set is divided into a training part (for training, 5027 reviews, ± 80%), a development set (for tuning, 549 reviews, ± 10%) and test part (for evaluation, 544 reviews, ± 10%). The dataset is balanced, which means positive and negative instances are evenly distributed. Additionally, training and test set contain reviews about different TV series (in order to avoid possible bias when evaluating). The dataset can be reached at : https://www.kaggle.com/datasets/djilax/allocine-french-movie-reviews

Reading and displaying the dataset

1. Import Pandas (refer to the official site https://pandas.pydata.org/docs/ if necessary) and use it to:
    a. Read the train dataset and displays the shape of it (number of lines, number of columns).
    b. Display the name of each column
    c. Display the sentiment and the review of the first entry in the dataset
    d. Read the dev dataset and displays the shape of it
    e. Read the test dataset and displays the shape of it

Generating tokens

2. Modify the code below to generate either all the tokens or only the verbs and nouns.

```
def preprocess_data( dataset, all_tokens=True ):
  num_reviews = dataset['review'].size
  print("#Reviews =", num_reviews)
  dataset_tok = []
  for i in range(num_reviews):
     clean_review = review_to_tokens(dataset['review'][i], all_tokens=all_tokens)
     dataset_tok.append(clean_review)
  for i, r in enumerate(dataset_tok[:2]):
     print( '\n', i, dataset['review'][i] )
     print(r)
  return dataset_tok
def review_to_tokens(raw_review, all_tokens=True):
   # Use Spacy to tokenize the data
   doc = nlp(raw_review)
```

3. Display the preprocessed train data, dev data, test data, with all tokens and with only verb and noun tokens.

Vectorizing and defining the vocabulary

4. BOW representation : import CountVectorizer from scikit learn and vectorize the train set and the dev set. Check the doc: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html
5. Display the 30 first obtained vocabulary of train and dev sets.
6. Restrict the vocabulary to the 5000 most frequent words, write the code for this restriction.

Training

7. We'll start with the simplest classifier, yet often performing well: **Naive Bayes.** Train the classifier and compute its performance on the dev set. (Training = fitting the model to the data (function *fit()*)).

8. Do the evaluation using a scorer function using classifier.score().

   Use classifier.predict() to dispaly other scores.

9. Display the instances that were classified badly, code below:

```
print('Misclassified examples: ')
count_err = 0
for i in range(len(pred)):
   if pred[i] != dev['sentiment'][i]:
      print( "\nGOLD=", dev['sentiment'][i], "PRED=",pred[i] , i, dev['review'][i])
      count_err += 1
```

10. Write the code to vectorize the data with all tokens into bigrams. Keep 'max_features = 500'. Then retrain and evaluate the classifier.
11. Try when keeping only verbs and nouns.
12. Try when keeping all tokens, but using both unigrams and bigrams.
13. Compare the results obtained with NB to the ones obtained with MaxEnt (logistic regression) (see the doc for logistic regression on scikit learn).
14. Compute the performance on the test set.
15. And now what do you conclude ?